Course Name: PCCCS495 – Advanced Programming

Project Title: Library Management System

Student Name: MEGHA SAHA

Roll Number: 12024052019053

CLASS ROLL NO:18

Instructor Name: SWAGATAM BASU,SUSOVON JANA,SOUMADIP BISWAS

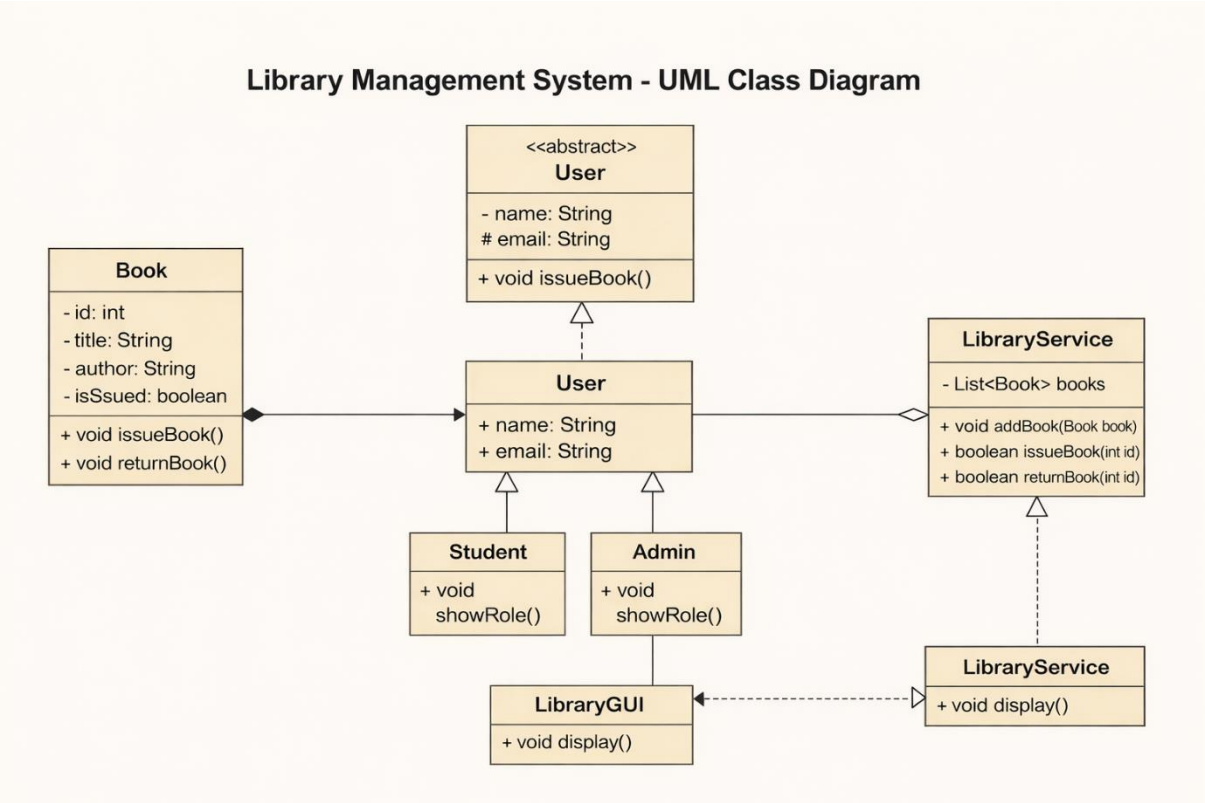
Semester: Spring 2026

PROBLEM STATEMENT:

In traditional library systems, managing books and users manually is time-consuming and prone to errors. Tasks such as adding books, issuing books, and tracking availability become inefficient when handled without automation.

This project aims to develop a **Library Management System using Java and OOP concepts** that simplifies these operations through a user-friendly graphical interface. The system allows users to add, view, issue, and return books efficiently, reducing manual workload and improving accuracy.

UML CLASS DIAGRAM:



Description of Major Classes

1. Book Class

- Stores book details (id, title, author, status)
- Methods: issueBook(), returnBook()

2. User Class (Abstract)

- Base class for users
- Contains common attributes

3. Student & Admin

- Inherit from User
- Represent different user roles

4. LibraryService

- Handles all logic:
  - Add book
  - Show books
  - Issue/return

5. LibraryGUI

- Provides graphical interface using Java Swing

Package Structure

```
src/
  ├── model/
  ├── service/
  ├── gui/
  └── Main.java
```

Interaction Flow

1. User enters book details in GUI
2. GUI sends data to LibraryService
3. LibraryService updates book list
4. Output is displayed in GUI

OOP CONCEPTS DEMONSTRATED

Abstraction

Where used:
User class is abstract

Why:
To define common structure for Student/Admin

Code:

```
public abstract class User {
   protected String name;
}
```

Explanation:
Abstract class hides implementation and provides a base structure.

Inheritance

Where used:
Student and Admin extend User

Code:

```
public class Student extends User {
}
```

Explanation:
Reuses common properties of User class.

Polymorphism

Where used:
Method overriding

Explanation:

Different classes behave differently using same method.

Encapsulation

Where used:
Book class uses private variables

Code:

```
private int id;
```

Explanation:
Data is protected using getters/setters.

IMPLEMENTATION HIGHLIGHTS

Snippet 1: Add Book

```
public void addBook(Book book) {
   books.add(book);
}
```

Justification:
Stores book in list for management.

Snippet 2: Issue Book

```
public void issueBook() {
   isIssued = true;
}
```

Justification:
Changes status of book.

Snippet 3: Return Book

```
public void returnBook() {
   isIssued = false;
}
```

Justification:
Marks book as available again.

TESTING & ERROR HANDLING

Test Cases

- Add valid book
- Show books
- Issue book
- Return book

Edge Cases

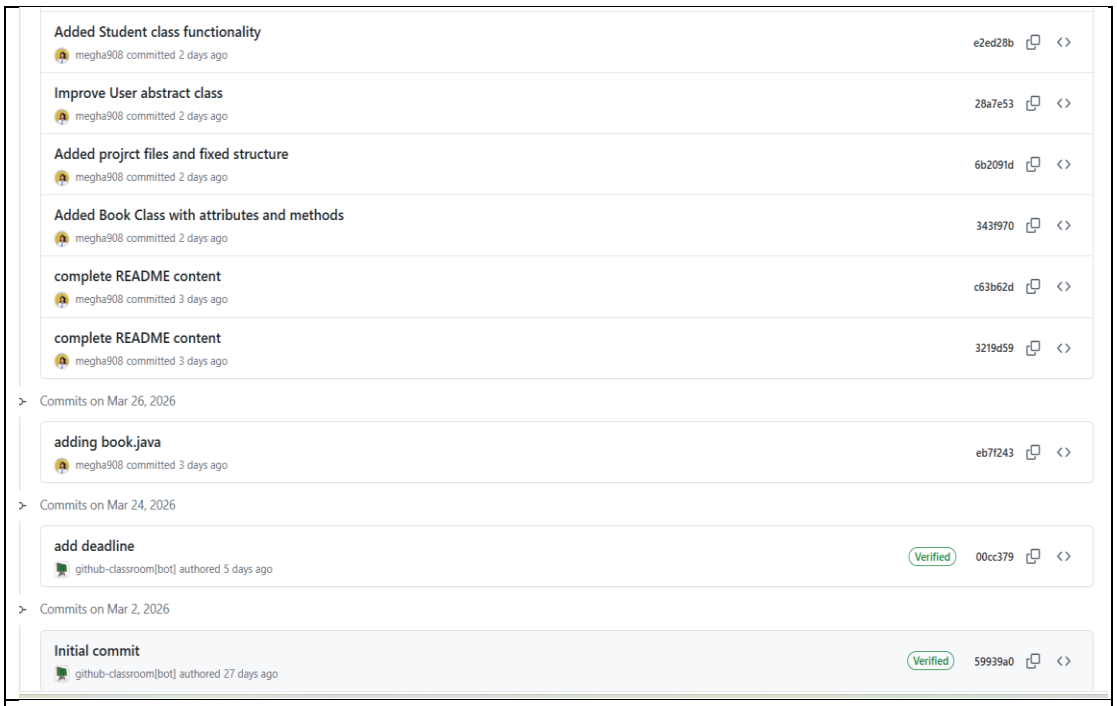
- Empty input fields
- Duplicate book ID

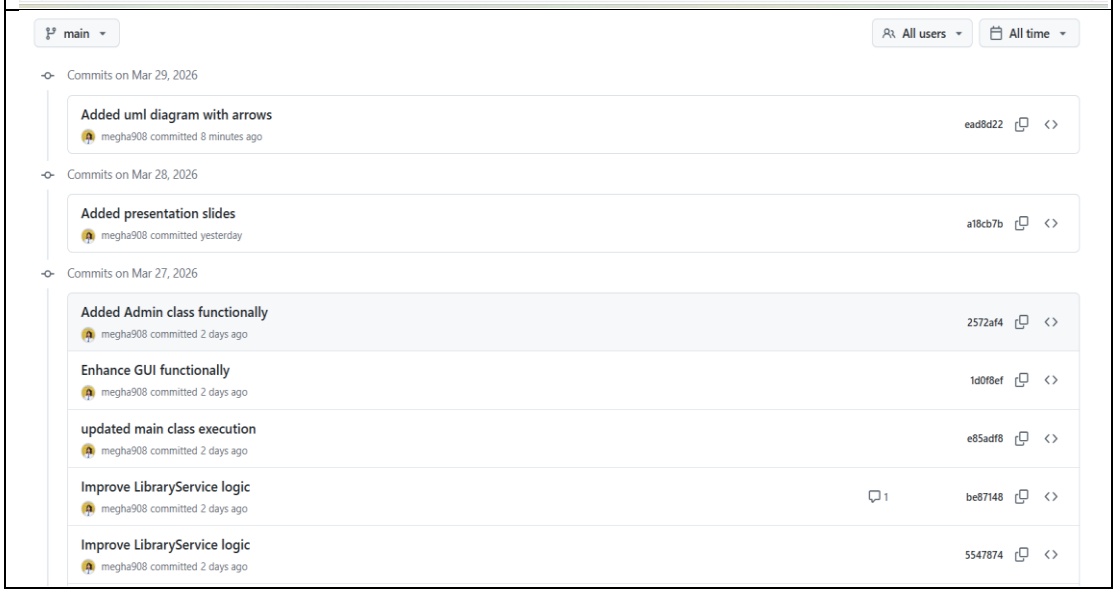
Failure Scenarios

- Invalid input
- No books available

GIT WORKFLOW:

COMMIT HISTROY:





EXPLANATION OF DEVELOPMENT PROGRESSION:

1. Initial Project Setup

The project began with creating the folder structure and initializing the Git repository. Basic directories such as model, service, and gui were created to organize the code.

Commit Message:
“Initial project structure setup”

2. Implementation of Book Class

The Book class was developed to store book-related data such as ID, title, author, and issue status. Methods for issuing and returning books were also implemented.

Commit Message:
“Added Book class with attributes and methods”

3. Implementation of User Abstraction

An abstract User class was created to represent common properties of users. This helped in applying abstraction and code reuse.

Commit Message:
“Implemented abstract User class”

4. Implementation of Inheritance (Student & Admin)

Two classes, Student and Admin, were developed by extending the User class. This demonstrated inheritance in the system.

Commit Message:
“Added Student and Admin classes using inheritance”

5. Development of LibraryService

The LibraryService class was implemented to manage core operations such as adding books, displaying books, issuing books, and returning books.

Commit Message:
“Created LibraryService for managing books”

6. Implementation of Core Functionalities

Core functionalities including add, show, issue, and return book operations were implemented and tested.

Commit Message:
“Implemented add, issue, and return book functionality”

7. GUI Development using Swing

A graphical user interface was developed using Java Swing to provide an interactive experience for users. Input fields and buttons were added for various operations.

Commit Message:
“Developed GUI using Java Swing”

8. Integration of GUI with Backend

The GUI was connected with the LibraryService class to enable real-time interaction between user inputs and system logic.

Commit Message:
“Connected GUI with backend service logic”

9. Error Handling and Validation

Basic validation was added to handle incorrect or empty inputs, ensuring the system behaves reliably.

Commit Message:
“Improved input handling and error checking”

10. Documentation and README Update

The README file was updated with project description, features, and instructions for running the application.

Commit Message:
“Updated README with project details”

11. Final Testing and Refinement

The project was tested thoroughly, and minor improvements were made to enhance performance and readability.

Commit Message:
“Final testing and code cleanup”

CONCLUSION & FUTURE SCOPE

Conclusion

The Library Management System was successfully developed using Java and OOP concepts. It provides a simple and efficient way to

manage library operations such as adding, issuing, and returning books.

Future Scope

- Add database integration
- Add login authentication
- Add search functionality
- Web-based version

APPENDIX:

COPPY OF THE INITIAL PROJECT REPORT:

Library Management System using Java

Problem Statement

Managing library records manually can be inefficient and time-consuming. Librarians must track books, issued records, and returns using manual registers or spreadsheets, which can lead to errors and misplaced records. Searching for books and checking availability also becomes difficult. The proposed Library Management System is a Java-based application designed to manage library data digitally. The system will allow users to add books, search books, issue books, and return books efficiently. By applying Object-Oriented Programming (OOP) principles, the system will organize data using classes and methods, making the system structured and easy to maintain. This project demonstrates the practical use of Java programming while improving efficiency and accuracy in managing library operations.

Target User

- Librarians
- Library staff
- Students who borrow books

Core Features

- Add new books to the system
- View all available books
- Search books by ID or title
- Issue books to users
- Return books to the library
- Update book availability status

OOP Concepts to be Used

- **Abstraction** – Library operations implemented through methods.
- **Inheritance** – Different types of books can extend a base Book class.
- **Polymorphism** – Methods behave differently based on object type.
- **Exception Handling** – Try–catch blocks handle invalid inputs.
- **Collections** – ArrayList used to store and manage book records.

Proposed Architecture Description

The system consists of multiple Java classes following object-oriented design. The Book class stores book information such as ID, title, author, and availability. The Library class manages operations like adding, searching, issuing, and returning books. The User class represents people who borrow books. The Main class provides a menu-driven interface where users interact with the system. Book records are stored using ArrayList from the Java Collections Framework, allowing dynamic storage and easy access to book data.

REVIEW MAIL RECEIVED:

PCCCS495 Proposal Review – APPROVED_MINOR

S soumadip.biswas@ieminternational.ai 10 Mar
To You

Dear MEGHA SAHA,

Project: Library Management System using Java
Decision: APPROVED_MINOR
Score: 80

Strengths:
The proposal clearly identifies a common and relevant problem that can be effectively addressed using a Java-based OOP solution. The problem statement is concise, and the proposed features (Add, View, Search, Issue, Return Books, Update Availability) directly tackle the stated inefficiencies. The architecture outlines a clear and understandable class structure (Main, Library, Book) that is appropriate for an introductory or intermediate OOP project, leveraging `ArrayList` for data storage. The explicit mention of key OOP concepts (Abstraction, Inheritance, Polymorphism, Encapsulation, Exception Handling, Collections) indicates a good understanding of foundational principles.

Improvements:
1. **Data Persistence:** The current architecture implies in-memory storage using `ArrayList`, meaning all data will be lost when the application closes. The proposal should address how book records will be persistently stored (e.g., using file I/O like serialization or CSV, or a basic database connection for a more advanced project).
2. **User/Member Management:** While the 'Issue Book' and 'Return Book' features involve 'users,' there's no mention of a `User` or `Member` class to manage library members, track who borrowed which book, or handle user-specific operations. Implementing this would significantly enhance the system's realism and demonstrate more complex OOP relationships.
3. **Concrete Application of Inheritance and Polymorphism:** While listed, the proposal doesn't provide specific examples of *how* `Inheritance` or `Polymorphism` would be applied within this particular system (e.g., different types of books inheriting from a base `Book` class, or different user roles). Providing such examples would strengthen the proposal's OOP depth.
4. **Robust Error Handling and Validation:** Beyond simply listing 'Exception Handling,' the proposal could benefit from detailing specific scenarios where exceptions would be handled (e.g., invalid book ID during search/issue, trying to issue an already issued book, or returning a book not checked out). Input validation should also be considered.

PCCCS495 Proposal Review – APPROVED_...

1. **Data Persistence:** The current architecture implies in-memory storage using `ArrayList`, meaning all data will be lost when the application closes. The proposal should address how book records will be persistently stored (e.g., using file I/O like serialization or CSV, or a basic database connection for a more advanced project).
2. **User/Member Management:** While the 'Issue Book' and 'Return Book' features involve 'users,' there's no mention of a `User` or `Member` class to manage library members, track who borrowed which book, or handle user-specific operations. Implementing this would significantly enhance the system's realism and demonstrate more complex OOP relationships.
3. **Concrete Application of Inheritance and Polymorphism:** While listed, the proposal doesn't provide specific examples of *how* `Inheritance` or `Polymorphism` would be applied within this particular system (e.g., different types of books inheriting from a base `Book` class, or different user roles). Providing such examples would strengthen the proposal's OOP depth.
4. **Robust Error Handling and Validation:** Beyond simply listing 'Exception Handling,' the proposal could benefit from detailing specific scenarios where exceptions would be handled (e.g., invalid book ID during search/issue, trying to issue an already issued book, or returning a book not checked out). Input validation should also be considered.
5. **Expanded Search Criteria:** Currently, search is limited to book ID or title. Expanding search capabilities to include author or availability status could be beneficial.

Regards,
Course Instructor